beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in beamer documents. Named overlay specifications are very handy both during edition and to manage complex and variable beamer overlay specifications. In particular, they allow to replace raw numbers in beamer `<...>` overlay specifications by logical identifiers. Demonstration files are available for download as part of the development repository. As a side effect, this is another solution to this `latex.org` forum query.

Contents

1 Implementation

Identify the internal prefix (LaTeX3 DocStrip convention).

```
1 ⟨@@=bnvs⟩
```

1.1 Package declarations

```
2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over expl.

```
8 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
9   \A \s* (?:
```

- 2 → V

```
    ( [^:]+? )
```

- 3, 4, 5 → A : Z? or A :: L?

```
  | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
```

- 6, 7 → ::(L:Z)?

```
  | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
```

- 8, 9 → :(Z::L)?

```
  | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
 )
 \s* \Z
}
```

1.3 Main scanner

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into LaTeX.

1.3.1 Grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters all have category other. We start with basic entities.

```
⟨spc⟩          ::= Horizontal white space characters
⟨lower⟩        ::= a lowercase letter
⟨upper⟩        ::= an uppercase letter
⟨digit⟩        ::= a decimal digit
⟨sign⟩         ::= '-' | '+'
⟨comma⟩        ::= ⟨spc⟩ ',' ⟨spc⟩
⟨alpha⟩        ::= ⟨lower⟩ | ⟨upper⟩ | '_'
⟨alnum⟩        ::= ⟨alpha⟩ | ⟨digit⟩
⟨index⟩        ::= ⟨sign⟩* ⟨digit⟩+
⟨number⟩       ::= ⟨sign⟩? ⟨significand⟩ ⟨exponent⟩?
⟨significand⟩  ::= ( '.'? ⟨digit⟩+ ) | ( ⟨digit⟩+ '.' ⟨digit⟩* )
⟨exponent⟩     ::= ( 'e' | 'E' ) ⟨sign⟩? ⟨digit⟩+
⟨other⟩        ::= ⟨number⟩ | '+' | '-' | '*' | '/'
```

`\c__bnvs_N_regex` Signed integer.

(*End of definition for* `\c__bnvs_N_regex`.)

```
\regex_const:Nn \c__bnvs_N_regex {
  [-+]* \d+
}
```

`\c__bnvs_dot_N_regex` Signed integer with a dot `.` just before.

(*End of definition for* `\c__bnvs_dot_N_regex`.)

```
\regex_const:Nn \c__bnvs_dot_N_regex {
  \. \ur{c__bnvs_N_regex}
}
```

\c__bnvs_A_dot_N_Z_regex Full signed integer with a dot . jus before.

(*End of definition for* \c__bnvs_A_dot_N_Z_regex.)

```
\regex_const:Nn \c__bnvs_A_dot_N_Z_regex {
  \A \ur { c__bnvs_dot_N_regex } \Z
}
```

\c__bnvs_A_i_Z_regex Signed integer with no capture groups.

(*End of definition for* \c__bnvs_A_i_Z_regex.)

```
\regex_const:Nn \c__bnvs_A_i_Z_regex { \A \ur{c__bnvs_N_regex} \Z }
```

\c__bnvs_dot_R_regex A possibly empty list of .⟨*short name*⟩ items representing a path. Integers are allowed as well.

```
\regex_const:Nn \c__bnvs_R_regex {
  \ur{c__bnvs_N_regex} | [[:alnum:]_]+
}
```

(*End of definition for* \c__bnvs_dot_R_regex.)

\c__bnvs_dot_R_regex A possibly empty list of .⟨*short name*⟩ items representing a path. Integers are allowed as well.

```
\regex_const:Nn \c__bnvs_dot_R_regex {
  \. \ur{c__bnvs_R_regex}
}
```

(*End of definition for* \c__bnvs_dot_R_regex.)

\c__bnvs_S_regex This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```
\regex_const:Nn \c__bnvs_S_regex {
  [[:alpha:]_][[:alnum:]_]*
}
```

(*End of definition for* \c__bnvs_S_regex.)

\c__bnvs_P_regex This regular expression is used for short alphanumerical path components with at least one letter.

```
\regex_const:Nn \c__bnvs_P_regex {
  \d*[[:alpha:]_][[:alnum:]_]*
}
```

(*End of definition for* \c__bnvs_P_regex.)

\c__bnvs_dot_P_regex This regular expression is used for short alphanumerical path components with at least one letter.

```
\regex_const:Nn \c__bnvs_dot_P_regex {
  \. \ur{c__bnvs_P_regex}
}
```

(*End of definition for* `\c__bnvs_dot_P_regex`.)

The lists between curly delimiters are used for ⟨*key*⟩=⟨*value*⟩ definitions, with optional values. Each ⟨*CrlItm*⟩ end with either a comma, an unbalanced closing delimiter or `\q_stop`. Here ⟨*Rhs*⟩ ends exactly the same.

```
⟨Crl⟩     ::= '{' ⟨CrlLst⟩ '}'
⟨CrlLst⟩  ::= ⟨spc⟩ ⟨CrlItm⟩ ( ⟨comma⟩ ⟨CrlItm⟩ )* ⟨spc⟩
⟨CrlItm⟩  ::= ⟨CrlEql⟩ | ⟨IKSPR⟩
⟨CrlEql⟩  ::= ( ⟨IKSPR⟩ ⟨equal⟩ )+ ⟨Rhs⟩
```

The lists between square delimiters are used for ⟨*key*⟩=⟨*value*⟩ definitions, with optional keys.

```
⟨Sqr⟩    ::= '[' ⟨SqrLst⟩ ']'
⟨SqrLst⟩ ::= ⟨spc⟩ ⟨SqrItm⟩ ( ⟨comma⟩ ⟨SqrItm⟩ )* ⟨spc⟩
⟨SqrItm⟩ ::= ⟨SqrEql⟩ | ⟨Rhs⟩
⟨SqrEql⟩ ::= ( ( ⟨IKSPR⟩ | ⟨index⟩ ) ⟨equal⟩ )+ ⟨Rhs⟩
```

For function calls,

```
⟨Raw⟩    ::= '"' '(' ⟨RawLst⟩ ')'
⟨RawLst⟩ ::= ( ⟨spc⟩ | ⟨Rlv⟩ | ⟨RawRnd⟩ | ⟨RawSqr⟩ | ⟨RawCrl⟩ | ⟨RawOtr⟩ )*
⟨RawRnd⟩ ::= '(' ⟨RawLst⟩ ')'
⟨RawSqr⟩ ::= '[' ⟨RawLst⟩ ']'
⟨RawCrl⟩ ::= '{' ⟨RawLst⟩ '}'
⟨RawOtr⟩ ::= ⟨comma⟩ | ⟨alnum⟩ | ⟨other⟩
```

1.3.2 Storage

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual "\", "{" and "}" before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use TEX groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

There is also a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

`\BNVS_xprt_clear:`
`\BNVS_xprt:n`
`\BNVS_scan_xprt_black:n`

```
\BNVS_xprt_clear:
\BNVS_xprt:n {⟨material⟩}
\BNVS_scan_xprt_black:n {⟨material⟩}
```

`\BNVS_xprt_clear:` resets the scan storage, called when starting a new TEX group dedicated to scanning. `\BNVS_xprt:n` is a utility function to store its argument in

the `xprted` variable. Automatically set by `\BNVS_begin_scan:...` functions. Used by `\BNVS_end_scan:`. The default implementation just raises. `\BNVS_xprt_grp:n` feeds the scan storage with ⟨*material*⟩, enclosed betwenn one pair of curly braces. `\BNVS_scan_xprt_black:n` is called just before exporting. The argument is enclosed inside `\exp_not:n{`⟨...⟩`}`. The default implementation just expands to its argument as is.

```
\cs_new:Npn \BNVS_xprt_clear: {
  \__bnvs_tl_clear:c { xprted }
}

\cs_new:Npn \BNVS_xprt:n #1 {
  \BNVS_xprt_stored:
  \__bnvs_tl_put_right:cn { xprted } { #1 }
}
```

`\BNVS_xprt_grp:n` `\BNVS_xprt_grp:n {`⟨*scanned*⟩`}`
`\BNVS_xprt_use:n` `\BNVS_xprt_use:n {`⟨*code:n*⟩`}`

`\BNVS_xprt_grp:n` exports its argument between curly braces. `\BNVS_xprt_use:n` puts back to the input stream the exported material so far, enclosed in one pair of braces after the instructions ⟨*code:n*⟩.

```
\cs_new:Npn \BNVS_xprt_use:n #1 {
  \BNVS_xprt_stored:
  \BNVS_tl_use:nv { #1 } { xprted }
}

\cs_new:Npn \BNVS_xprt_grp:n #1 {
  \BNVS_xprt:n { { #1 } }
}
```

`\BNVS_scan_xprt_prefix:n` `\BNVS_scan_xprt_prefix:n {`⟨*xprt prefix:w*⟩`}`

Set the function `\BNVS_xprt_prefix:w` to expand to `\exp_not:n { `⟨*xprt prefix:w*⟩`}`. This is used to define `\BNVS_scan_xprt:n`, when the arguments must be scanned prior to knowing which function ⟨*wrapped:w*⟩ fits.

```
\cs_new:Npn \BNVS_xprt_prefix:w {}

\cs_new:Npn \BNVS_scan_xprt_prefix:n #1 {
  \cs_set:Npn \BNVS_xprt_prefix:w { \exp_not:n { #1 } }
}
```

`\BNVS_xprt_wrap:n` `\BNVS_xprt_wrap:n {`⟨*xprt:n*⟩`}`
`\BNVS_xprt_wrap_crl:n` `\BNVS_xprt_wrap_crl:n {`⟨*xprt:n*⟩`}`
`\BNVS_xprt_wrap_raw:n` `\BNVS_xprt_wrap_raw:n {`⟨*xprt:n*⟩`}`

Set the function `\BNVS_scan_xprt:n`. The first variant exports only black prepared material. The second variant exports material between curly brackets. The third variant exports material a is.

```
\cs_new:Npn \BNVS_xprt_wrap:n #1 {
  \cs_set:Npn \BNVS_scan_xprt:n ##1 {
    \tl_if_blank:nF { ##1 } {
      \BNVS_xprt_prefix:w
```

and \exp_not:n{⟨*xprt:n*⟩}.

```
      \exp_not:n { #1 }
    }
  }
}
\BNVS_xprt_wrap:n { #1 }

\cs_new:Npn \BNVS_xprt_wrap_raw:n #1 {
  \cs_set:Npn \BNVS_scan_xprt:n ##1 {
    \BNVS_xprt_prefix:w
    \tl_if_blank:nF { ##1 } {
```

and \exp_not:n{⟨*xprt:n*⟩}.

```
      \exp_not:n { #1 }
    }
  }
}

\cs_new:Npn \BNVS_xprt_wrap_crl:n #1 {
  \cs_set:Npn \BNVS_scan_xprt:n ##1 {
    \BNVS_xprt_prefix:w
```

and \exp_not:n{⟨*xprt:n*⟩}.

```
    \exp_not:n { { #1 } }
  }
}
```

\BNVS_xprt_store:n
\BNVS_xprt_stored:

\BNVS_xprt_store:n {⟨*something*⟩}
\BNVS_xprt_stored:

\BNVS_xprt_store:n synchronously cumulates its argument in the `stored` variable. The function \BNVS_xprt_stored: is executed both at the beginning of the function \BNVS_xprt:n and when a scan group ends. It exports the contents of the stored ⟨*material*⟩, when non empty, to the `xprted` variable as \:n{⟨*material*⟩} instruction and cleans the `stored` variable.

```
\cs_new:Npn \BNVS_xprt_stored: {
  \__bnvs_tl_if_empty:cF { stored } {
    \BNVS_tl_use:nv {
      \__bnvs_tl_clear:c { stored }
      \BNVS_xprt:n { \:n }
      \BNVS_xprt_grp:n
    } { stored }
  }
}

\cs_new:Npn \BNVS_xprt_store:n #1 {
  \tl_if_empty:nF { #1 } {
    \__bnvs_tl_put_right:cn { stored } { #1 }
  }
}
```

\BNVS_next_set:n \BNVS_next_set:n {⟨*next:*⟩}
\BNVS_next_use:n \BNVS_next_use:n {⟨*code:*⟩}

\BNVS_next_set:n stores ⟨*next:*⟩ to be executed later by \BNVS_next_use:n, just after ⟨*code:*⟩. Typically, \BNVS_next_set:n is used in a scan group closed by ⟨*code:*⟩. \BNVS_end_scan_next: ends the scan and executes ⟨*next:*⟩ code.

```
\BNVS_tl_new:c { next }
\cs_new:Npn \BNVS_next_set:n #1 {
  \__bnvs_tl_set:cn { next } {
    \exp_not:n {
      #1
    }
  }
}

\cs_new:Npn \BNVS_next_use:n #1 {
  \BNVS_tl_last_unbraced:nv { #1 } { next }
}
```

1.3.3 Scanner

Spaces are ignored arround separators and delimiters.

\BNVS_scan:nnnnnnnw \BNVS_scan:nnnnnnnw {⟨*preamble:*⟩}
\BNVS_scan:nnw {⟨*closed:*⟩} {⟨*comma:*⟩} {⟨*one_colon:*⟩} {⟨*colons:*⟩} {⟨*stopped:*⟩}
{⟨*postamble:w*⟩} ⟨*input stream*⟩

Start a scanning branch section by executing the ⟨*preamble:*⟩ instructions. The five next arguments are function bodies used to define the corresponding branch functions. The ⟨*postamble:w*⟩ instructions are finally executed. They are expected to scan the input stream and terminate by calling one of the branch functions just defined.

```
\cs_new:Npn \BNVS_scan:nnnnnnnw #1 #2 #3 #4 #5 #6 #7 {
  #1
  \cs_set:Npn \BNVS_scanned_close: { #2 }
  \cs_set:Npn \BNVS_scanned_comma: { #3 }
  \cs_set:Npn \BNVS_scanned_one_colon: { #4 }
  \cs_set:Npn \BNVS_scanned_colons: { #5 }
  \cs_set:Npn \BNVS_scanned_q_stop: { #6 }
  #7
```

There must be absolutely no code here. No hooking allowed either.

```
}

\cs_new:Npn \BNVS_scan:nnw #1 #2 {
  \BNVS_scan:nnnnnnnw {
    \BNVS_begin_scan:
```

```
    #1
  }
    { \BNVS_end_scan_next: \BNVS_scanned_close: }
    { \BNVS_end_scan_next: \BNVS_scanned_comma: }
    { \BNVS_end_scan_next: \BNVS_scanned_one_colon: }
    { \BNVS_end_scan_next: \BNVS_scanned_colons: }
    { \BNVS_end_scan_next: \BNVS_scanned_q_stop: }
    { #2 }
}
```

`\BNVS_begin_scan:` `\BNVS_begin_scan:`

Start a scanning group.

```
\cs_new:Npn \BNVS_begin_scan: {
  \BNVS_scan_will_begin:
  \BNVS_begin:
  \BNVS_scan_did_begin:
}

\cs_new:Npn \BNVS_scan_will_begin: {
  \BNVS_xprt_stored:
}

\cs_new:Npn \BNVS_scan_did_begin: {
  \BNVS_xprt_clear:
  \BNVS_scan_xprt_prefix:n {}
  \BNVS_next_set:n {}
}
```

`\BNVS_end_scan:n` `\BNVS_end_scan:n {`⟨*material-x*⟩`}`
`\BNVS_end_scan:` `\BNVS_end_scan:`
`\BNVS_end_scan_next:` `\BNVS_end_scan_next:`

One of these functions must be called to close the TEX scan group. `\BNVS_end_scan_next:` calls `\BNVS_end_scan:` and what was stored as next instructions before closing the group. `\BNVS_end_scan:` call `\BNVS_end_scan:n` on what was exported before closing the group (at this level), once prepared by `\BNVS_scan_xprt:n`... Notice that ⟨*material-x*⟩ is `x`-expanded.

```
\cs_new:Npn \BNVS_end_scan:n #1 {
  \exp_args:NNx \BNVS_end: \BNVS_xprt:n {
    \BNVS_scan_xprt:n { #1 }
  }
}

\cs_new:Npn \BNVS_end_scan: {
  \BNVS_xprt_use:n {
    \BNVS_end_scan:n
  }
}

\cs_new:Npn \BNVS_end_scan_next: {
  \BNVS_next_use:n { \BNVS_end_scan: }
}
```

\BNVS_begin_scan:nnnnnnn `\BNVS_begin_scan:nnnnnnn {⟨xprt:n⟩} {⟨next:⟩}`
`{⟨closed:⟩} {⟨comma:⟩} {⟨one colon:⟩} {⟨colons:⟩} {⟨stopped:⟩}`

Start a scanning group with all branch functions definitions. ⟨*xprt:n*⟩ is processed by \BNVS_xprt_wrap:n.

One shot utility only used by \BNVS_begin_scan:nnnnnnn and friends.

```
\cs_new:Npn \BNVS_scan_begun:nnnnnnn #1 #2 #3 #4 #5 #6 #7 {
  \BNVS_xprt_clear:
  \BNVS_xprt_wrap:n { #1 }
  \BNVS_next_set:n {
    #2
  }
  \cs_set:Npn \BNVS_scanned_close: {
    #3
  }
  \cs_set:Npn \BNVS_scanned_comma: {
    #4
  }
  \cs_set:Npn \BNVS_scanned_one_colon: {
    #5
  }
  \cs_set:Npn \BNVS_scanned_colons: {
    #6
  }
  \cs_set:Npn \BNVS_scanned_q_stop: {
    #7
  }
}

\cs_new:Npn \BNVS_begin_scan:nnnnnnn {
  \BNVS_begin:
  \BNVS_scan_begun:nnnnnnn
}
```

\BNVS_begin_scan:nn `\BNVS_begin_scan:nn {⟨xprt:n⟩} {⟨next:n⟩}`

OBSOLETE. Call \BNVS_begin_scan:nnnnnnn with default arguments that end the scan group and call the eponym branch function.

```
\cs_new:Npn \BNVS_begin_scan:nnX #1 #2 {
  \BNVS_begin_scan:nnnnnnn { #1 } { #2 }
    { \BNVS_end_scan: \BNVS_scanned_close: }
    { \BNVS_end_scan: \BNVS_scanned_comma: }
    { \BNVS_end_scan: \BNVS_scanned_one_colon: }
    { \BNVS_end_scan: \BNVS_scanned_colons: }
    { \BNVS_end_scan: \BNVS_scanned_q_stop: }
}
```

In the sequel, ⟨*input:w*⟩ is expected to read the ⟨*input stream*⟩ until a \q_stop token that is gobbled.

\c__bnvs_close_rnd_regex Round closing delimiter.

```
\regex_const:Nn \c__bnvs_close_rnd_regex { \s* %(
  \) \s* }
```

(*End of definition for* `\c__bnvs_close_rnd_regex`.)

`\c__bnvs_close_sqr_regex` Square closing delimiter.

```
\regex_const:Nn \c__bnvs_close_sqr_regex { \s* %[
  \] \s* }
```

(*End of definition for* `\c__bnvs_close_sqr_regex`.)

`\c__bnvs_close_crl_regex` Round closing delimiter.

```
\regex_const:Nn \c__bnvs_close_crl_regex { \s* %{
  \} \s* }
```

(*End of definition for* `\c__bnvs_close_crl_regex`.)

`\BNVS_scan_close:Fw` `\BNVS_scanned_close:N` `\BNVS_scanned_close:`

`\BNVS_scan_close:Fw {`⟨*else:*⟩`}` ⟨*input stream*⟩
`\BNVS_scanned_close:N` ⟨*clositer*⟩

When a closing delimiter is scanned, call `\BNVS_scanned_close:N` with that delimiter as argument. Otherwise execute ⟨*else:*⟩ code.

`\BNVS_scanned_close:N` is a branch function called when the closing delimiter ⟨*clositer*⟩ has just been scanned. After it manages eventual errors in balancing delimiters, it calls the branch function `\BNVS_scanned_close:` used many times. When redefined (for example by `\__bnvs_scan_delimited:NNnFw`), the `\BNVS_scanned_close:` function does not forward to the eponym function, instead it must know where to go next.

```
\group_begin:
\char_set_catcode_group_begin:N <
\char_set_catcode_group_end:N >
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scan_close:Fw #1 <
  \peek_regex_replace_once:NnTF \c__bnvs_close_rnd_regex < > <
    \BNVS_scanned_close:N%(
    )
  > <
    \peek_regex_replace_once:NnTF \c__bnvs_close_sqr_regex < > <
      \BNVS_scanned_close:N%[
      ]
    > <
      \peek_regex_replace_once:NnTF \c__bnvs_close_crl_regex < > <
        \BNVS_scanned_close:N%{
        }
      > <
        #1
      >
    >
  >
>
\group_end:

\cs_set:Npn \BNVS_scanned_close:N #1 {
```

```
    \BNVS_error:n { Unexpected~delimiter~`#1' }
    \BNVS_scanned_close:
  }
```

Do nothing operation at the top level.

```
  \cs_set:Npn \BNVS_scanned_close: {
  }
```

`\c__bnvs_comma_regex` Comma as a separator.

```
\regex_const:Nn \c__bnvs_comma_regex { \s* , \s* }
```

(*End of definition for* `\c__bnvs_comma_regex`.)

`\__bnvs_scan_comma:Fw` `\BNVS_scanned_comma:`

`\__bnvs_scan_comma:Fw {⟨else:⟩} ⟨input stream⟩`

On scanning a comma from the head of the ⟨*input stream*⟩, calls `\BNVS_scanned_comma:` branch function, otherwise execute ⟨*else:*⟩ instructions. Beware, it gobbles spaces.

```
\BNVS_new:cpn { scan_comma:Fw } #1 {
```

`\peek_regex_remove:NnTF` is buggy in L3 programming layer <2025-01-18>.

```
    \peek_regex_replace_once:NnTF \c__bnvs_comma_regex {} {
      \BNVS_scanned_comma:
    } {
      #1
    }
```

There must be absolutely no code here. No hooking allowed either.

```
  }
```

Do nothing operation at the top level.

```
  \cs_set:Npn \BNVS_scanned_comma: {
  }
```

`\c__bnvs_colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

```
\regex_const:Nn \c__bnvs_colons_regex { \s* :(:*) \s* }
```

(*End of definition for* `\c__bnvs_colons_regex`.)

`\__bnvs_scan_colon:Fw` `\BNVS_scanned_one_colon:` `\BNVS_scanned_colons:`

`\__bnvs_scan_colon:Fw {⟨else:⟩} ⟨input stream⟩`

On scanning a standalone colon from the ⟨*input stream*⟩, calls the branch function `\BNVS_scanned_one_colon:`. On scanning multiple colons from the ⟨*input stream*⟩, calls the `\BNVS_scanned_colons:` branch function. In other cases, execute the ⟨*else:*⟩ instructions.

```
\cs_new:Npn \BNVS_scanned_colons:n #1 {
  \tl_if_empty:nTF { #1 } {
    \BNVS_scanned_one_colon:
  } {
```

```
    \BNVS_scanned_colons:
  }
}
\BNVS_new:cpn { scan_colon:Fw } #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_colons_regex { { \1 } } {
    \BNVS_scanned_colons:n
  } {
    #1
  }
}
```

Do nothing operation at the top level.

```
\cs_set:Npn \BNVS_scanned_one_colon: {
}
```

Do nothing operation at the top level.

```
\cs_set:Npn \BNVS_scanned_colons: {
}
```

`\BNVS_scan_q_stop:F` `\BNVS_scanned_q_stop:`

`\BNVS_scan_q_stop:F {⟨else:⟩} ⟨input stream⟩`

On scanning a `\q_stop` from the head of the ⟨*input stream*⟩, calls the `\BNVS_scanned_q_stop:` branch function, otherwise execute ⟨*else:*⟩ instructions.

```
\cs_new:Npn \BNVS_scan_q_stop:F {
  \peek_meaning_remove:NTF \q_stop {
    \BNVS_scanned_q_stop:
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}
```

Do nothing operation at the top level.

```
\cs_set:Npn \BNVS_scanned_q_stop: {
}
```

`\__bnvs_scan_delimiter:Fw`

`\__bnvs_scan_delimiter:Fw {⟨else:⟩} ⟨input stream⟩`

Medley of the above,

```
\BNVS_new:cpn { scan_delimiter:Fw } #1 {
  \BNVS_scan_close:Fw {
    \__bnvs_scan_comma:Fw {
      \__bnvs_scan_one_colon:Fw {
        \__bnvs_scan_colons:Fw {
          \__bnvs_scan_stop:F {
            #1
          }
        }
      }
    }
  }
}
```

1.3.4 Delimiters

We assume that delimiters are paired and balanced. Nevertheless, some errors are catched and properly managed.

\BNVS_scan_close:nnnNn `\BNVS_scan_close:nnnNn {⟨xprt prefix:w⟩} {⟨xprt:n⟩} {⟨next:⟩} ⟨close⟩ {⟨postamble:w⟩}`

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using ⟨*postamble:w*⟩. Implementation detail: we begin a scanning group that will be closed on scanning the ⟨*close*⟩ delimiter, or a different closing delimiter, allways at the same level. The ⟨*postamble:w*⟩ function scans the ⟨*input stream*⟩, and is expected to end on a closing delimiter at the current level by sending \BNVS_scanned_close:N. This is the normal instruction flow.

```
\cs_new:Npn \BNVS_scan_close:nnnNn #1 #2 #3 #4 #5 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \cs_set:Npn \BNVS_scanned_close:N ##1 {
      \tl_if_eq:nnF { #4 } { ##1 } {
```

If the closing delimiter scanned is not the expected one, raise.

```
        \BNVS_error:n { Unexpected~`##1'~instead~of~`#4'. }
      }
```

Anyways, branch here.

```
      \BNVS_end_scan_next:
    }
    \BNVS_scan_xprt_prefix:n { #1 }
    \BNVS_xprt_wrap_raw:n { #2 }
    \BNVS_next_set:n { #3 }
  }
```

Close branch. End the group and execute ⟨*next:*⟩ instructions. Intermediate groups should be closed appropriately with the right definition of branch functions.

```
    { \BNVS_end_scan_next: }
```

Any other branch function raises and forwards to \BNVS_scanned_close:,

```
    { \BNVS_error:n { Missing~`#4'. } \BNVS_scanned_close: }
    { \BNVS_error:n { Missing~`#4'. } \BNVS_scanned_close: }
    { \BNVS_error:n { Missing~`#4'. } \BNVS_scanned_close: }
```

or \BNVS_scanned_q_stop: after the group ends when \q_stop is just scanned.

```
    {
      \BNVS_error:n { Missing~`#4'. }
      \BNVS_end_scan_next:
      \BNVS_scanned_q_stop:
    } {
```

One group level for the contents.

```
      #5
    }
}
```

\c__bnvs_open_rnd_regex Round closing delimiter.

```
\regex_const:Nn \c__bnvs_open_rnd_regex { \s* \( %(
  \s* }
```

(*End of definition for* `\c__bnvs_open_rnd_regex`.)

\c__bnvs_open_sqr_regex Square closing delimiter.

```
\regex_const:Nn \c__bnvs_open_sqr_regex { \s* \[ %]
  \s* }
```

(*End of definition for* `\c__bnvs_open_sqr_regex`.)

\c__bnvs_open_crl_regex Round closing delimiter.

```
\regex_const:Nn \c__bnvs_open_crl_regex { \s* \{ %}
  \s* }
```

(*End of definition for* `\c__bnvs_open_crl_regex`.)

\BNVS_scan_inside:w `\BNVS_scan_inside:w` ⟨*input stream*⟩

Scan balanced material taking only delimiters into account. Used to parse calls to functions.

```
\cs_new:Npn \BNVS_scan_inside:n #1 {
  \BNVS_xprt:n { #1 }
  \BNVS_scan_inside:w
}

\group_begin:
\char_set_catcode_group_begin:N <
\char_set_catcode_group_end:N >
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scan_inside:w <
  \peek_regex_replace_once:NnTF \c__bnvs_open_rnd_regex < > <
    \BNVS_scan_close:nnnNn
      < > < ##1 > < \BNVS_xprt:n < ) > \BNVS_scan_inside:w > %(
      ) < \BNVS_xprt:n < ( > \BNVS_scan_inside:w >
  > <
    \peek_regex_replace_once:NnTF \c__bnvs_open_sqr_regex < > <
      \BNVS_scan_close:nnnNn
        < > < ##1 > < \BNVS_xprt:n < ] > \BNVS_scan_inside:w >%[
        ] < \BNVS_xprt:n < [ > \BNVS_scan_inside:w >
    > <
      \peek_regex_replace_once:NnTF \c__bnvs_open_crl_regex < > <
        \BNVS_scan_close:nnnNn
          < > < ##1 > < \BNVS_xprt:n < } > \BNVS_scan_inside:w >%{
          } < \BNVS_xprt:n < { > \BNVS_scan_inside:w >
      > <
        \BNVS_scan_close:Fw <
          \BNVS_scan_q_stop:F <
            \BNVS_scan_inside:n
          >
```

```
        >
      >
    >
  >
>
\group_end:
```

1.3.5 Comma separated lists

\BNVS_scan_csl_item:w `\BNVS_scan_csl_item:w` ⟨*input stream*⟩

Scan a single item in a comma separated list. The item ends at the first top level comma, the first unbalanced delimiter if one is found, or when a `\q_stop` is found. The `stored` variable is fed with material that cannot be processed by ⟨*scan:Fw*⟩ function.

```
\cs_set:Npn \BNVS_scan_csl_item:w {
  \BNVS_scan_special:TFw {
    \BNVS_scan_csl_item:w
  } {
    \__bnvs_scan_comma:Fw {
      \BNVS_scan_close:Fw {
        \BNVS_scan_q_stop:F {
          \__bnvs_scan_csl_other:Nw
        }
      }
    }
  }
}

\cs_new:Npn \BNVS_scan_special:TFw #1 #2 {
  \BNVS_error:n { \BNVS_scan_special:TFw~undefined~at~top~level. }
}

\BNVS_new:cpn { scan_csl_other:Nw } #1 {
  \BNVS_xprt_store:n { #1 }
  \BNVS_scan_csl_item:w
}
```

\BNVS_scan_csl:nnnw `\BNVS_scan_csl:nnnw` {⟨*closed:*⟩} {⟨*stopped:*⟩} {⟨*postamble:w*⟩} ⟨*input stream*⟩

This function scans the comma separated list of items, regardless the enclosing delimiters. It begins a scanning TEX group that is ended by a call to `\BNVS_scanned_close:`, executing ⟨*closed:*⟩, or `\BNVS_scanned_q_stop:`, executing ⟨*stop:*⟩. The scanned material is then exported between braces. On scanning a comma, a new group is created. The exported material is empty for an empty list, for input `a` it is `\:nn{a}{}`, for input `a,b` it is `\:nn{a}{\:nn{b}{}}`, and so on.

```
\cs_new:Npn \BNVS_scan_csl:nnnw #1 #2 #3 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
```

```
    \BNVS_scan_xprt_prefix:n { \:nn }
    \BNVS_xprt_wrap:n { { ##1 } { } }
  }
```

Close branch.

```
    { \BNVS_end_scan: #1 }
    {
      \BNVS_xprt_use:n { \tl_if_empty:nTF } {
```

Comma branch, the first item is empty, skip it.

```
        #3
      } {
```

Comma branch, the first item is not empty, there will be another item, possibly empty.

```
        \BNVS_xprt_wrap:n { { ####1 } }
        \BNVS_end_scan:
        \BNVS_scan:nnnnnnnw {
          \BNVS_begin_scan:
          \BNVS_xprt_wrap:n { { ####1 } }

        }
          { \BNVS_end_scan: \BNVS_scanned_close: }
          { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
          { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
          { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
          { \BNVS_end_scan: \BNVS_scanned_q_stop: }
        {
          \BNVS_scan_csl:nnnw {
```

Close branch.

```
            \BNVS_xprt_use:n { \tl_if_empty:nTF } {
              \BNVS_end_scan:
              \BNVS_xprt:n { { } }
            } {
              \BNVS_end_scan:
            }
            \BNVS_scanned_close:
          } {
```

Stop branch.

```
            \BNVS_xprt_use:n { \tl_if_empty:nTF } {
            \BNVS_end_scan:
              \BNVS_xprt:n { { } }
            } {
              \BNVS_end_scan:
            }
            \BNVS_scanned_q_stop:
          } {
            #3
          }
        }
      }
    }
    { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
    { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
    { \BNVS_end_scan: #2 }
```

B

By default, the list contains only one item.

```
  { #3 }
}
```

1.3.6 VAZL atoms

Here is a preliminar grammar for ranges or values:

```
⟨V⟩        ::= ⟨blnc⟩
⟨A⟩        ::= ⟨blnc⟩
⟨Z⟩        ::= ⟨blnc⟩
⟨L⟩        ::= ⟨blnc⟩
⟨VAZL⟩     ::= ⟨V⟩ | ⟨AZL⟩ | ⟨ALZ⟩
⟨AZL⟩      ::= ⟨A⟩? ':'  ⟨Z⟩ ( "::" ⟨L⟩? )?
⟨ALZ⟩      ::= ⟨A⟩? "::" ⟨L⟩ ( ':'  ⟨Z⟩? )?
⟨VAZLRnd⟩  ::= '(' ⟨VAZLLst⟩ ')'
⟨VAZLLst⟩  ::= ⟨spc⟩ ⟨VAZL⟩ ( ⟨comma⟩ ⟨VAZL⟩ )* ⟨spc⟩
⟨Rlv⟩      ::= '?' ⟨VAZLRnd⟩
⟨blnc⟩     ::= ( ⟨spc⟩ | ⟨IKSPR⟩ | ⟨Rnd⟩ | ⟨Sqr⟩ | ⟨Crl⟩ | ⟨Rlv⟩ | ⟨Raw⟩ |
    ↪  ⟨other⟩ )*
```

\BNVS_scan_VAZL:nw `\BNVS_scan_VAZL:nw {⟨postamble:w⟩} ⟨input stream⟩`

Scan the input stream for a VAZL atom. The scanning process will end on the first comma, the first unbalanced closing delimiter (at the same level) or \q_stop. The {⟨postamble:w⟩} instructions are used to scan the ⟨input stream⟩, they end with one of \BNVS_scanned_close:, \BNVS_scanned_comma: or \BNVS_scanned_q_stop:.

Utility function: either a ⟨A⟩:⟨Z⟩:: or a ⟨A⟩::⟨L⟩: has just been scanned. The argument is a ⟨postamble:w⟩ set of instructions.

```
\cs_set:Npn \BNVS_scan_Z_or_L:nw #1 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \BNVS_xprt_wrap:n { { ##1 } }
  } { \BNVS_end_scan: \BNVS_scanned_close: }
    { \BNVS_end_scan: \BNVS_scanned_comma: }
```

No colon separator allowed here. We simply ignore them and scan balanced material.

```
    { \BNVS_error:n { No~`:'~here,~ignored. } #1 }
    { \BNVS_error:n { No~`::'~here,~ignored. } #1 }
    { \BNVS_end_scan: \BNVS_scanned_q_stop: }
    { #1 }
}
```

Utility function: a ⟨A⟩: has just been scanned. The argument is a ⟨postamble:w⟩ set of instructions.

```
\cs_set:Npn \BNVS_scan_ZL:nw #1 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \BNVS_xprt_wrap:n { { ##1 } }
  }
```

We export the third argument which is empty. The same for comma and \q_stop below.

```
    { \BNVS_end_scan: \BNVS_xprt:n { { } } \BNVS_scanned_close: }
    { \BNVS_end_scan: \BNVS_xprt:n { { } } \BNVS_scanned_comma: }
    {
      \BNVS_end_scan:
      \BNVS_error:n { Missing~`:'. }
      \BNVS_scan_Z_or_L:nw { #1 }
    }
    { \BNVS_end_scan: \BNVS_scan_Z_or_L:nw { #1 } }
    { \BNVS_end_scan: \BNVS_xprt:n { { } } \BNVS_scanned_q_stop: }
    { #1 }
}
```

Utility function: a ⟨*A*⟩:: has just been scanned. The argument is a ⟨*postamble:w*⟩ set of instructions.

```
\cs_set:Npn \BNVS_scan_LZ:nw #1 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \BNVS_xprt_wrap:n { { ##1 } }
  }
```

We export the third argument which is empty. The same for comma and \q_stop below.

```
    { \BNVS_end_scan: \BNVS_xprt:n { { } } \BNVS_scanned_close: }
    { \BNVS_end_scan: \BNVS_xprt:n { { } } \BNVS_scanned_comma: }
    { \BNVS_end_scan: \BNVS_scan_Z_or_L:nw { #1 } }
    {
      \BNVS_end_scan:
      \BNVS_error:n { No~`::'~here. }
      \BNVS_scan_Z_or_L:nw { #1 }
    }
    { \BNVS_end_scan: \BNVS_xprt:n { { } } \BNVS_scanned_q_stop: }
  { #1 }
}

\cs_set:Npn \BNVS_scan_VAZL:nw #1 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \BNVS_scan_xprt_prefix:n { \V:w }
    \BNVS_xprt_wrap:n { { ##1 } }
  }
    { \BNVS_end_scan: \BNVS_scanned_close: }
    { \BNVS_end_scan: \BNVS_scanned_comma: }
    {
      \BNVS_scan_xprt_prefix:n { \AZL:w }
      \BNVS_end_scan:
      \BNVS_scan_ZL:nw { #1 }
```

```
    }
    {
      \BNVS_scan_xprt_prefix:n { \ALZ:w }
      \BNVS_end_scan:
      \BNVS_scan_LZ:nw { #1 }
    }
    { \BNVS_end_scan: \BNVS_scanned_q_stop: }
  { #1 }
}
```

`\BNVS_scan_csl_VAZL:nnnnw` `\BNVS_scan_csl_VAZL:nnw {⟨xprt prefix:w⟩} {⟨close:⟩} {⟨q_stop:⟩} {⟨postamble:w⟩} ⟨input stream⟩`

Scan the input stream for a comma separated list of VAZL atom.

```
\cs_new:Npn \BNVS_scan_csl_VAZL:nnnnw #1 #2 #3 #4 {
  \BNVS_scan:nnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \BNVS_scan_xprt_prefix:n { #1 }
    \BNVS_xprt_wrap:n { { ##1 } }
  }
  { \BNVS_end_scan: #2 } { } { } { } { } { \BNVS_end_scan: #3 }
  { \BNVS_scan_csl:nnnw
    { \BNVS_scanned_close: }
    { \BNVS_scanned_q_stop: }
    { \BNVS_scan_VAZL:nw { #4 } }
  }
}
```

1.3.7 Assignments, function calls and identifiers

In the following grammar, ⟨*S*⟩ stands for a short identifier, ⟨*P*⟩ stands for a path component and ⟨*R*⟩ stands for a relative component.

```
⟨ISRCE⟩    ::= ⟨ISR⟩ | ( ⟨ISR⟩ ⟨Call⟩ ) | ( ⟨ISR⟩ ⟨MirEql⟩ ⟨Rhs⟩ )
⟨ISR⟩      ::= ( ( ⟨S⟩? '!' )? ⟨S⟩ ) | '.' ⟨P⟩ ) ⟨R⟩*
⟨S⟩        ::= ⟨alpha⟩ ⟨alnum⟩*
⟨P⟩        ::= ⟨digit⟩* ⟨S⟩
⟨R⟩        ::= ( '.' ⟨P⟩ ) | ( '.' ⟨N⟩ ) | ⟨n⟩
⟨n⟩        ::= '[' ⟨spc⟩ ⟨blnc⟩ ⟨spc⟩ ']'
⟨Call⟩     ::= '(' ⟨spc⟩ ⟨blnc⟩ ⟨spc⟩ ')'
⟨MirEql⟩   ::= ⟨spc⟩ ( '=' | '+=' | '-=' | '/=' | '*=' | ) ⟨spc⟩
⟨Rhs⟩      ::= ⟨VAZLRhs⟩ | '?(' ⟨VAZLRhs⟩ ')' | '"(' ⟨VAZLRhs⟩ ')'
⟨VAZLRhs⟩  ::= ⟨VAZL⟩ | ⟨VAZLRnd⟩ | ⟨Crl⟩ | ⟨Sqr⟩
```

`\BNVS_scan_R_crl:nw` `\BNVS_scan_R_crl:nw {⟨next:⟩} ⟨input stream⟩`

Exports between curly braces what is exported by `\BNVS_scan_R:nw`.

```
\cs_new:Npn \BNVS_scan_R_crl:nw #1 {
```

```
  \BNVS_scan:nnw {
%    \begin{macrocode}
% \begin{BNVS.gobble}
⟨/!debug⟩
% \end{BNVS.gobble}
    \BNVS_xprt_wrap_crl:n { ##1 }
    \BNVS_next_set:n { #1 }
  } { \BNVS_scan_R:nw { \BNVS_end_scan_next: } }
}
```

`\BNVS_scanned_P:nww` `\BNVS_scanned_P:nww {⟨next:⟩} {⟨P⟩} ⟨input stream⟩`

Exports `\PR:w{⟨P⟩}`, scan a relative component and calls `\BNVS_scan_R_crl:nw`.

```
\cs_new:Npn \BNVS_scanned_P:nww #1 #2 {
  \BNVS_xprt:n { \PR:w { #2 } }
  \BNVS_scan_R_crl:nw { #1 }
}
```

`\BNVS_scanned_N:nww` `\BNVS_scanned_N:nww {⟨next:⟩} {⟨N⟩} ⟨input stream⟩`

Exports `\NR:w{⟨N⟩}`, scan a relative component and calls `\BNVS_scan_R_crl:nw`. ⟨*N*⟩ is normalized.

```
\cs_new:Npn \BNVS_scanned_N:nww #1 #2 {
  \BNVS_xprt:n { \NR:w }
  \exp_args:Ne \BNVS_xprt_grp:n { \int_eval:n { #2 } }
  \BNVS_scan_R_crl:nw { #1 }
}
```

`\BNVS_scan_R:nw` `\BNVS_scan_R:nw {⟨next:⟩} ⟨input stream⟩`

Calls `\BNVS_scanned_P:nww` or `\BNVS_scanned_N:nww`, export the scanned material between curly braces and execute ⟨*next:*⟩ code.

```
\cs_new:Npn \BNVS_scan_R:nw #1 {
  \peek_charcode_remove:NTF . {
```

If we scan a "`.`", scan ⟨*P*⟩ material and call `\BNVS_scanned_P:nww` or scan ⟨*N*⟩ material and call `\BNVS_scanned_N:nww`, both with ⟨*next:*⟩ instructions.

```
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
      \BNVS_scanned_P:nww { #1 }
    } {
      \peek_regex_replace_once:NnTF \c__bnvs_N_regex { { \0 } } {
        \BNVS_scanned_N:nww { #1 }
      } {
```

Otherwise execute ⟨*next:*⟩ code then reinsert the dot.

```
        #1
        .
      }
    }
  } {
    \peek_charcode_remove:NTF [%]
    {
```

If we scan a "[", scan balanced material and execute ⟨*next:*⟩ code.

```
      \BNVS_scan_close:nnnNn { \nR:w } { { ##1 } }
```

Scan for a rhs in a csl.

```
      { \BNVS_scan_R_crl:nw { #1 } } %[
      ]
      { \BNVS_scan_blnc:w }
    } {
```

Otherwise execute ⟨*else:w*⟩ code.

```
      #1
    }
  }
}
```

`\c__bnvs_IKS_regex` The start of a qualified dotted name, with three capture groups.

(*End of definition for* `\c__bnvs_IKS_regex`.)

```
\regex_const:Nn \c__bnvs_IKS_regex {
```

1: the optional frame ⟨*id*⟩ with

2: the optional exclamation mark

```
  (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: followed by a non empty short name.

```
  ( \ur{c__bnvs_S_regex} )
}
```

`\__bnvs_scan_ISRCE:TFw` `\__bnvs_scan_ISRCE:TFw {⟨yes:⟩} {⟨no:⟩} ⟨input stream⟩`

Scan an identifier, a function call or an assignment, then execute ⟨*yes:*⟩ on success or ⟨*no:*⟩ otherwise. Starting with an empty `xprted` variable, here are some examples of the expected output for standalone identifiers

- `I!S` → `\ISR:w{I}{S}{}`
- `I!S.A` → `\ISR:w{I}{S}{\PR:w{A}{}}`
- `I!S.A.B` → `\ISR:w{I}{S}{\PR:w{A}{\PR:w{B}{}}}`
- `I!S.+-123` → `\ISR:w{I}{S}{\NR:w{-123}{}}`
- `I!S[1+2]` → `\ISR:w{I}{S}{\nR:w{1+2}{}}`

and not standalone (calls and assignments)

- `I!S(`⟨`...`⟩`)` → `\ISR:wn{I}{S}{}{`⟨`...`⟩`}`
- `I!S?=`⟨*Rhs*⟩ → `\ISR:wnn{I}{S}{}{?}{`⟨*Rhs*⟩`}`

As we do not want to overscan the same input, we must keep track of the identifier before we can know whether the input stream contains standalone material or not.

```
\BNVS_new:cpn { scan_ISRCE:TFw } #1 #2 {
  \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex { { \1 } { \2 } { \3 } } {
```

We begin a scan group, the true branch will be executed at the end. By default we start on a standalone identifier, and we will change when necessary. `\BNVS_scanned_IKS_RCE:nww` will feed the `scanned` variable.

```
    \BNVS_scan:nnw {
      \BNVS_next_set:n { \BNVS_scan_CE:nw { #1 } }
```

By default we start on a standalone identifier,

```
      \BNVS_scan_xprt_prefix:n { \ISR:w }
      \BNVS_xprt_wrap:n { ##1 }
    } {
      \BNVS_scanned_IKS_RCE:nww { \BNVS_end_scan_next: }
    }
  } {
```

We begin a scan group, the true branch will be executed at the end. By default we start on a standalone identifier, and we will change when necessary. `\BNVS_scanned_P:nww` will feed the `xprted` variable.

```
    \peek_charcode_remove:NTF . {
      \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
          \BNVS_scan:nnw {
        \BNVS_next_set:n { \BNVS_scan_CE:nw { #1 } }
```

By default we start on a standalone identifier,

```
        \BNVS_scan_xprt_prefix:n { \ISR:w }
        \exp_args:Ne \BNVS_xprt_wrap:n {
          { \__bnvs_tl_use:c I }
          { \__bnvs_tl_use:c S }
          \exp_not:n { { ##1 } }
        }
      }
      { \BNVS_scanned_P:nww { \BNVS_end_scan_next: } }
      } {
        \peek_regex_replace_once:NnTF \c__bnvs_N_regex { { \0 } } {
```

We begin a scan group, the true branch will be executed at the end. By default we start on a standalone identifier, and we will change when necessary. The `xprted` variable will be properly fed by `\BNVS_scanned_N:nww`.

```
          \BNVS_scan:nnw {
            \BNVS_next_set:n { \BNVS_scan_CE:nw { #1 } }
            \BNVS_scan_xprt_prefix:n { \ISR:w }
            \exp_args:Ne \BNVS_xprt_wrap:n {
              { \__bnvs_tl_use:c I }
              { \__bnvs_tl_use:c S }
              \exp_not:n { { ##1 } }
            }
          } {
            \BNVS_scanned_N:nww { \BNVS_end_scan_next: }
          }
        } {
```

Choose the false branch and inject back the removed dot.

```
            #2
            .
          }
        }
      } {
```

Choose also the false branch.

```
        #2
      }
    }
}
```

\BNVS_scanned_IKS_RCE:nww `\BNVS_scanned_IKS_RCE:nww {⟨next:⟩} {⟨I⟩} {⟨K⟩} {⟨S⟩} ⟨input stream⟩`

Utility function only called once by `\__bnvs_scan_ISRCE:TFw`.

```
\cs_new:Npn \BNVS_scanned_IKS_RCE:nww #1 #2 #3 #4 {
  \tl_if_empty:nTF { #3 } {
```

No frame identifier given, use the current value. Store the short name, store in the appropriate variable.

```
    \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
  } {
```

Frame identifier given. Proceed as before mutatis mutandis.

```
    \__bnvs_tl_set:cn I { #2 }
    \__bnvs_tl_clear_new:c { S(#2) }
    \__bnvs_tl_set:cn { S(#2) } { #4 }
  }
```

Export the frame identifier and the short name: the `xprted` variable contains `{⟨I⟩}{⟨S⟩}`.

```
  \BNVS_tl_use:Nv \BNVS_xprt_grp:n I
  \BNVS_xprt_grp:n { #4 }
```

Scan next relative components, then a call or an assignment if any.

```
  \BNVS_scan:nnw {
    \BNVS_next_set:n { \BNVS_scan_CE:nw { #1 } }
    \BNVS_xprt_wrap_crl:n { ##1 }
  } {
    \BNVS_scan_R:nw { \BNVS_end_scan_next: }
  }
}
```

\c__bnvs_mireql_regex Assignment operators.

(*End of definition for* `\c__bnvs_mireql_regex`.)

```
\regex_const:Nn \c__bnvs_mireql_regex {
  \s*
```

1: the optional modifier.

```
  ( [-+/*?] )?

  =\s*
}
```